EXPT.NO: 9	SIMULATION OF DPSK, QPSK AND QAM GENERATION AND DETECTION SCHEMES
DATE:	

A)DPSK GENERATION AND DETECTION USING MATLAB

AIM:

To generate and detect a Differential Phase Shift Keying (DPSK) signal using MATLAB.

HARDWARE/SOFTWARE REQUIREMENTS:

- Personal Computer (PC) or Laptop
- MATLAB (R2018 or later)
- Windows/ Linux Operating system

THEORY:

Differential Phase Shift Keying (DPSK) is a digital modulation technique in which the information is represented by the change in phase of the carrier signal rather than its absolute phase. A binary '1' causes a phase change of 180°, while a binary '0' causes no phase change. Since DPSK uses phase differences between consecutive symbols, it does not require a coherent reference signal at the receiver, making the receiver simpler than that of BPSK.

PROCEDURE:

1. Open MATLAB and create a new script.

2. Type the MATLAB program for DPSK generation and detection.

3. Save the file with a suitable name (e.g., dpsk.m).

4. Click Run or press F5.

5. Observe the transmitted DPSK symbols in the Command Window.

6. Observe the recovered binary sequence.

7. Verify that the recovered data matches the transmitted data.

8. Record the output and conclude the experiment.

MATLAB CODE:

```
clc;
clear;
close all;

%% Parameters
Tb = 1;              % Bit duration
N = 8;               % Number of bits
samples = 100;       % Samples per bit
fc = 5;              % Carrier frequency

%% Input Binary Data
data = [1 0 1 1 0 0 1 0];

%% Generate Square Wave Input Signal
input_signal = [];

for i = 1:length(data)
   if data(i)==1
      input_signal = [input_signal ones(1,samples)];
   else
      input_signal = [input_signal zeros(1,samples)];
   end
end

t = 0:Tb/samples:N*Tb-Tb/samples;

%% Carrier Signal
carrier = sin(2*pi*fc*t);

%% DPSK Modulation
phase = 0;
dpsk = [];

for i = 1:length(data)

   if data(i)==1
      phase = phase + pi;     % Phase reversal
   end
```

```
    tt = (i-1)*Tb:Tb/samples:i*Tb-Tb/samples;

    if mod(round(phase/pi),2)==0
        wave = sin(2*pi*fc*tt);
    else
        wave = -sin(2*pi*fc*tt);
    end

    dpsk = [dpsk wave];
end

%% DPSK Demodulation
received = [];

prev_phase = 0;

for i=1:length(data)

    index = (i-1)*samples+1:i*samples;

    current = dpsk(index);

    corr = sum(current.*carrier(index));

    if corr>=0
        current_phase = 0;
    else
        current_phase = 1;
    end

    if i==1
        bit = current_phase;
    else
        bit = xor(current_phase,prev_phase);
    end

    received = [received bit];

    prev_phase = current_phase;
end

%% Recovered Square Wave
```

```
demod_signal = [];

for i=1:length(received)

   if received(i)==1
      demod_signal=[demod_signal ones(1,samples)];
   else
      demod_signal=[demod_signal zeros(1,samples)];
   end

end

%% Plots

figure;

subplot(4,1,1)
plot(t,input_signal,'LineWidth',2)
grid on
axis([0 N -0.5 1.5])
title('Input Square Wave')
xlabel('Time')
ylabel('Amplitude')

subplot(4,1,2)
plot(t,carrier,'r','LineWidth',1.5)
grid on
title('Carrier Wave')
xlabel('Time')
ylabel('Amplitude')

subplot(4,1,3)
plot(t,dpsk,'k','LineWidth',1.5)
grid on
title('DPSK Modulated Wave')
xlabel('Time')
ylabel('Amplitude')

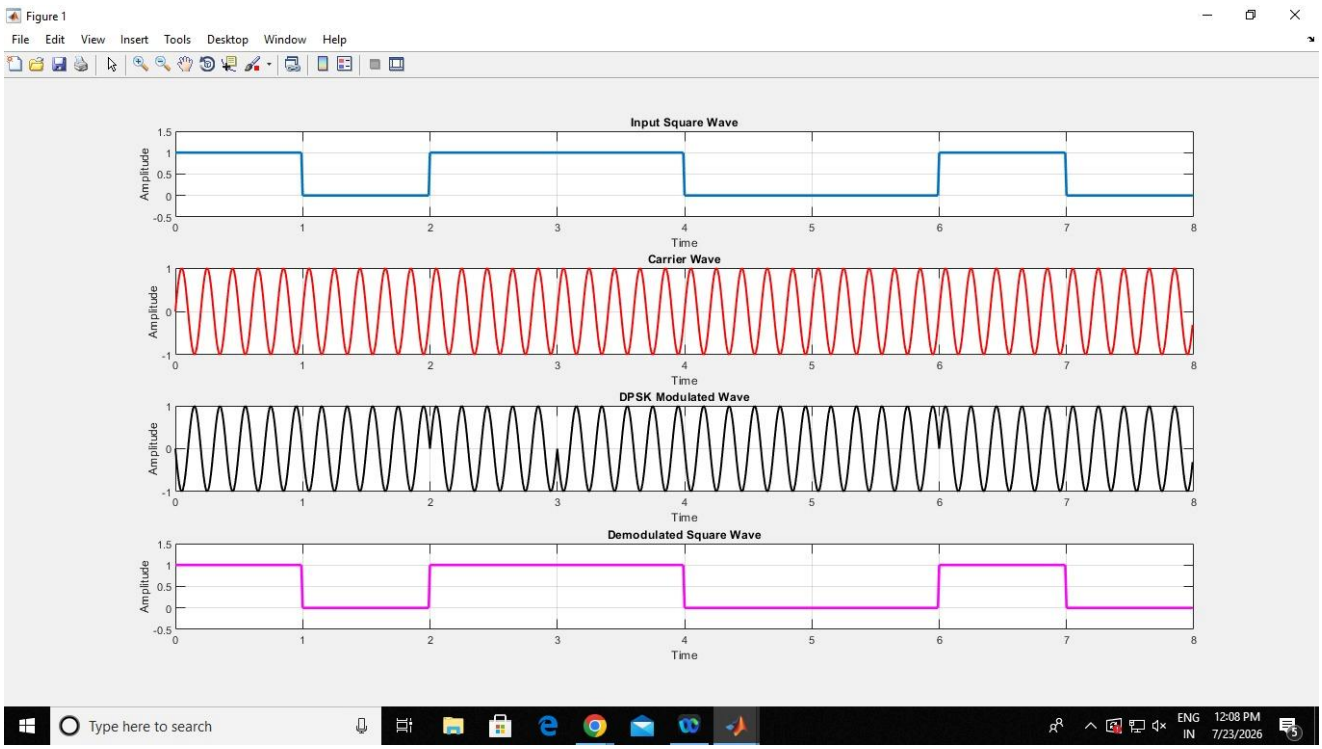
subplot(4,1,4)
plot(t,demod_signal,'m','LineWidth',2)
grid on
axis([0 N -0.5 1.5])
title('Demodulated Square Wave')
```

```
xlabel('Time')
ylabel('Amplitude')

%% Display Results
disp('Original Data:')
disp(data)

disp('Recovered Data:')
disp(received)
```

OUTPUT:



APPLICATIONS:

- Satellite communication
- Wireless communication systems
- RFID systems
- Optical fiber communication
- Digital radio communication

ADVANTAGES:

- Does not require carrier phase synchronization.
- Simpler receiver design.
- Better performance in channels with phase ambiguity.
- Easy to implement.

DISADVANTAGES:

- Slightly higher error probability than coherent BPSK.
- Errors may propagate to the next bit due to differential encoding.
- Less power efficient than coherent PSK.

RESULT:

The DPSK signal was successfully generated and detected using MATLAB. The transmitted binary sequence was correctly recovered at the receiver, demonstrating successful differential modulation and demodulation.

B)QPSK GENERATION AND DETECTION USING MATLAB

AIM:

To generate and detect a Quadrature Phase Shift Keying (QPSK) signal using MATLAB.

HARDWARE/SOFTWARE REQUIREMENTS:

- Personal Computer (PC) or Laptop
- MATLAB (R2018 or later)
- Windows/ Linux Operating system

THEORY:

Quadrature Phase Shift Keying (QPSK) is a digital modulation technique in which 2 bits are transmitted per symbol by changing the phase of the carrier signal. Since each symbol represents two bits, QPSK is more bandwidth-efficient than Binary Phase Shift Keying (BPSK).

The four possible phase values are:

Input Bits	Phase (°)	Symbol
00	225°	-1 − j
01	135°	-1 + j
10	315°	1 − j
11	45°	1 + j

The received signal is detected by checking the signs of the real (In-phase) and imaginary (Quadrature) components.

PROCEDURE:

1. Open MATLAB and create a new script file.

2. Enter the MATLAB program for QPSK generation and detection in the script editor.

3. Define the input binary data sequence.

4. Group the input bits into pairs, since QPSK maps 2 bits to 1 symbol.

5. Convert each bit pair into the corresponding In-phase (I) and Quadrature (Q) components to generate the QPSK symbols.

6. Execute the program to obtain the QPSK-modulated signal.

7. Plot the constellation diagram to observe the four QPSK symbol locations.

8. Perform QPSK demodulation by checking the signs of the real and imaginary parts of the received symbols.

9. Recover the transmitted binary data from the detected symbols.

10. Compare the recovered bit sequence with the original input sequence to verify correct modulation and demodulation.

11. Record the transmitted symbols, received bits, and constellation diagram as the simulation output.

MATLAB CODE:

```
clc;
clear;
close all;

%% Input Data (Even number of bits)
data = [0 1 0 1 1 1 0 0];

figure(1)
stem(data,'filled','LineWidth',2);
grid on;
title('Information Before Transmitting');
xlabel('Bit Number');
ylabel('Amplitude');
axis([0 length(data)+1 -0.2 1.5]);

%% NRZ Conversion
data_NRZ = 2*data - 1;

%% Serial to Parallel
sp_data = reshape(data_NRZ,2,[]);

%% Parameters
br = 1e6;
f = br;
```

```
T = 1/br;
t = T/99:T/99:T;

y = [];
y_in = [];
y_qd = [];

%% QPSK Modulation
for i = 1:size(sp_data,2)

   in_phase = sp_data(1,i)*cos(2*pi*f*t);
   quad_phase = sp_data(2,i)*sin(2*pi*f*t);

   y_in = [y_in in_phase];
   y_qd = [y_qd quad_phase];
   y = [y in_phase + quad_phase];

end

Tx_sig = y;
tt = T/99:T/99:(length(Tx_sig))*T/99;

figure(2)

subplot(3,1,1)
plot(tt,y_in,'LineWidth',2)
grid on
title('In-Phase Component')

subplot(3,1,2)
plot(tt,y_qd,'LineWidth',2)
grid on
title('Quadrature Component')

subplot(3,1,3)
plot(tt,Tx_sig,'k','LineWidth',2)
grid on
title('QPSK Modulated Signal')

%% QPSK Demodulation

Rx_sig = Tx_sig;
Rx_data = [];
```

```
for i = 1:size(sp_data,2)

   r = Rx_sig((i-1)*length(t)+1:i*length(t));

   z1 = r.*cos(2*pi*f*t);
   z2 = r.*sin(2*pi*f*t);

   I = trapz(t,z1)*(2/T);
   Q = trapz(t,z2)*(2/T);

   if I > 0
      b1 = 1;
   else
      b1 = 0;
   end

   if Q > 0
      b2 = 1;
   else
      b2 = 0;
   end

   Rx_data = [Rx_data b1 b2];

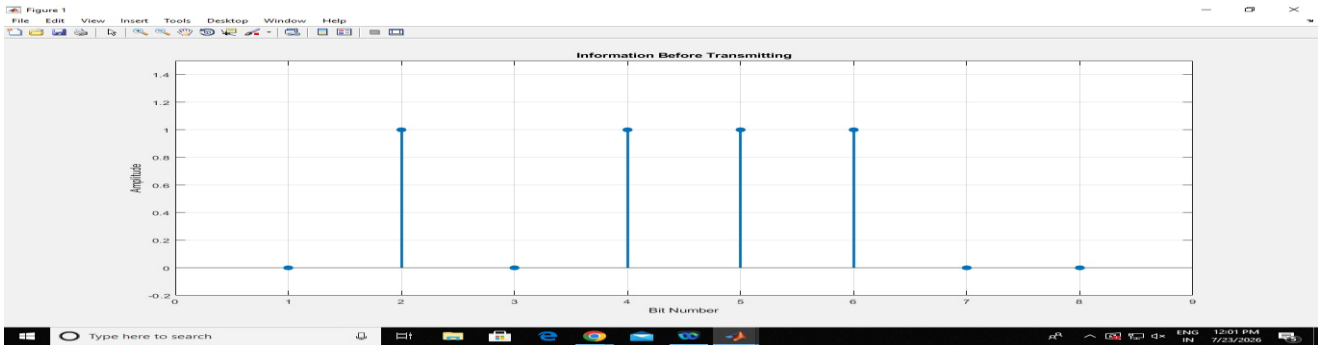
end

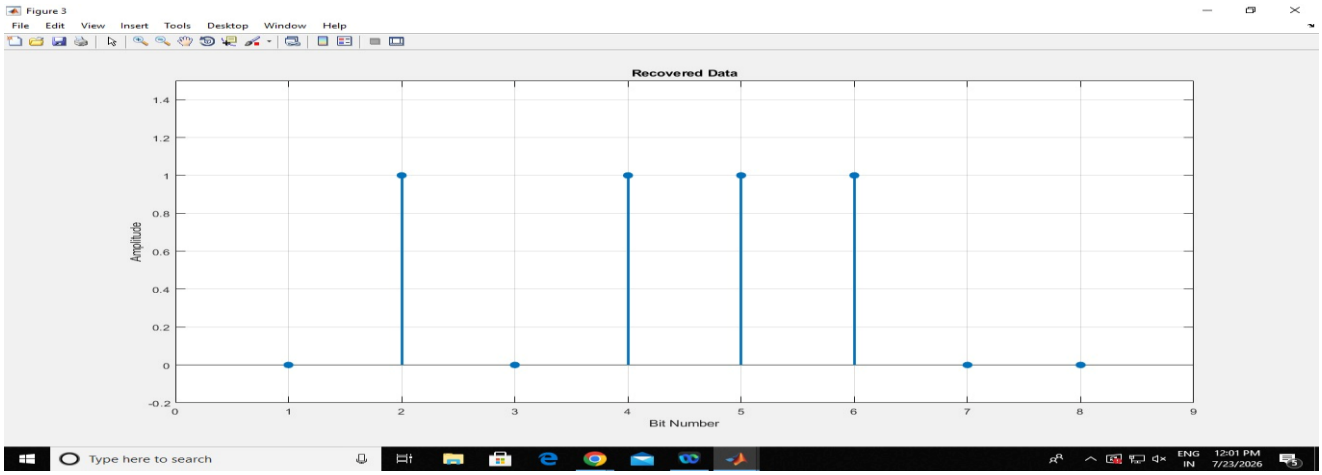
figure(3)
stem(Rx_data,'filled','LineWidth',2)
grid on
title('Recovered Data')
xlabel('Bit Number')
ylabel('Amplitude')
axis([0 length(Rx_data)+1 -0.2 1.5])

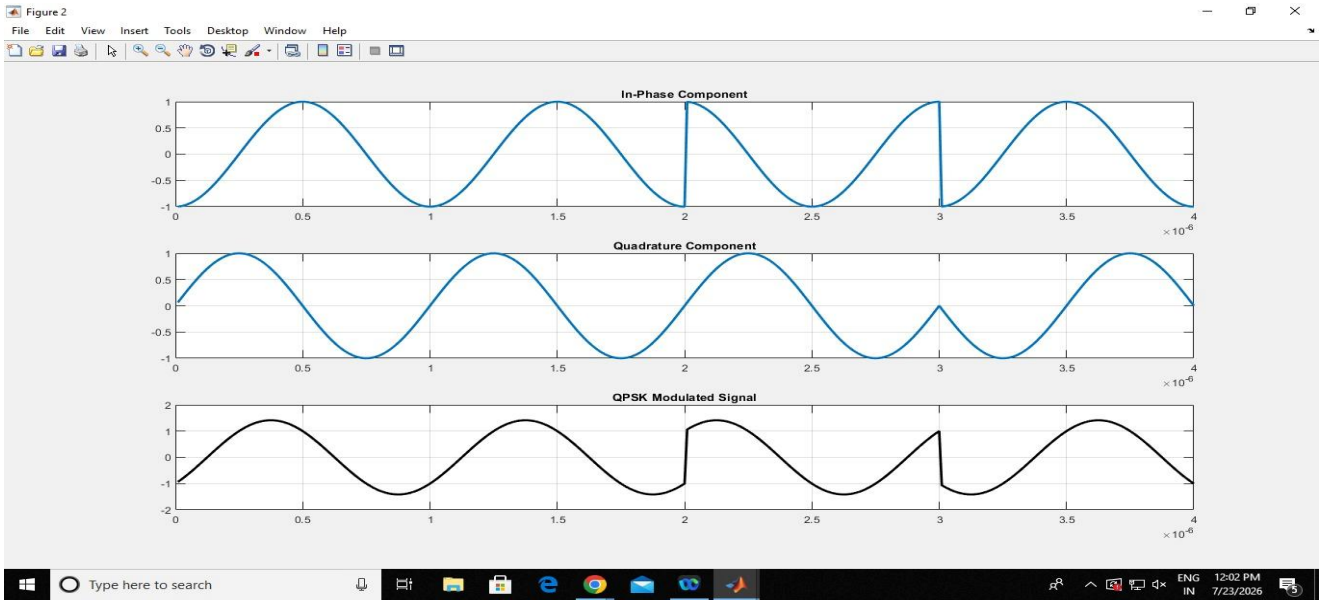
disp('Transmitted Data:')
disp(data)

disp('Received Data:')
disp(Rx_data)
```

OUTPUT:







APPLICATIONS:

- 4G LTE and 5G communication
- Wi-Fi systems

- Satellite communication
- Digital television (DVB)
- Wireless communication systems

ADVANTAGES:

- Transmits 2 bits per symbol.
- Better bandwidth efficiency than BPSK.
- Good noise immunity.
- Widely used in modern digital communication systems.

DISADVANTAGES:

- More complex than BPSK.
- Requires accurate carrier phase synchronization.
- Performance degrades in noisy channels.

RESULT:

The QPSK modulation and demodulation were successfully implemented using MATLAB. The transmitted binary sequence was correctly recovered at the receiver, demonstrating the successful generation and detection of the QPSK signal.

C)QAM GENERATION AND DETECTION USING MATLAB

AIM:

To generate and detect a Quadrature Amplitude Modulated (QAM) signal using MATLAB.

HARDWARE/SOFTWARE REQUIREMENTS:

- Personal Computer (PC) or Laptop
- MATLAB (R2018 or later)
- Windows/ Linux Operating system

THEORY:

Quadrature Amplitude Modulation (QAM) is a digital modulation technique that transmits data by varying both the amplitude and the phase of the carrier signal. It combines two carriers that are 90° out of phase, known as the In-phase (I) and Quadrature (Q) components. In 16-QAM, each symbol represents 4 bits, making it more bandwidth-efficient than PSK techniques.

PROCEDURE:

1. Open MATLAB and create a new script.

2. Type the MATLAB code for QAM generation and detection.

3. Save the file as qam.m.

4. Click Run or press F5.

5. Observe the transmitted QAM symbols.

6. Observe the recovered binary sequence.

7. Display the QAM constellation diagram.

8. Verify that the received data matches the transmitted data.

9. Record the output and conclude the experiment.

MATLAB CODE:

```
clc;
clear;
close all;

%% Parameters
N = 1000;            % Number of bits
sps = 20;            % Samples per symbol
fc = 5;              % Carrier frequency
EbNo = 10;           % Eb/No in dB

%% Generate random bits
tx_bits = randi([0 1],1,N);

% Make even number of bits
if mod(length(tx_bits),2)~=0
   tx_bits(end+1)=0;
end

%% Group bits into pairs
bits = reshape(tx_bits,2,length(tx_bits)/2)';

%% QAM Mapping (Gray coding)
I = zeros(1,size(bits,1));
Q = zeros(1,size(bits,1));

for k=1:size(bits,1)

   if isequal(bits(k,:),[0 0])
      I(k)=1; Q(k)=1;

   elseif isequal(bits(k,:),[0 1])
      I(k)=-1; Q(k)=1;

   elseif isequal(bits(k,:),[1 1])
      I(k)=-1; Q(k)=-1;

   else
```

```
      I(k)=1; Q(k)=-1;
   end

end

% Normalize power
I = I/sqrt(2);
Q = Q/sqrt(2);

%% Upsampling
I_up = repelem(I,sps);
Q_up = repelem(Q,sps);

t = (0:length(I_up)-1)/sps;

%% Carrier
carrier_I = cos(2*pi*fc*t);
carrier_Q = sin(2*pi*fc*t);

%% 4-QAM Modulation
tx_signal = I_up.*carrier_I - Q_up.*carrier_Q;


%% AWGN Channel
signal_power = mean(tx_signal.^2);

noise_power = signal_power/(10^(EbNo/10));

noise = sqrt(noise_power)*randn(size(tx_signal));

rx_signal = tx_signal + noise;


%% Demodulation
I_rec = rx_signal .* carrier_I;
Q_rec = -rx_signal .* carrier_Q;


% Integrate over symbol duration
I_sample=[];
Q_sample=[];

for k=1:sps:length(I_rec)-sps+1
```

```
   I_sample=[I_sample mean(I_rec(k:k+sps-1))];
   Q_sample=[Q_sample mean(Q_rec(k:k+sps-1))];

end


%% Constellation
figure;
scatter(I_sample,Q_sample,'filled')
grid on
axis([-1.5 1.5 -1.5 1.5])
title('Constellation after Demodulation')
xlabel('In-phase')
ylabel('Quadrature')


%% Decision
rx_bits=[];

for k=1:length(I_sample)

   if I_sample(k)>0 && Q_sample(k)>0
      b=[0 0];

   elseif I_sample(k)<0 && Q_sample(k)>0
      b=[0 1];

   elseif I_sample(k)<0 && Q_sample(k)<0
      b=[1 1];

   else
      b=[1 0];

   end

   rx_bits=[rx_bits b];

end


%% BER
rx_bits=rx_bits(1:length(tx_bits));
```

```
BER=sum(tx_bits~=rx_bits)/length(tx_bits);

disp(['BER = ',num2str(BER)])


%% Plot Bits

figure;

subplot(2,1,1)
stairs(tx_bits(1:40),'LineWidth',2)
grid on
title('Transmitted Bits (first 40)')
ylim([-0.5 1.5])


subplot(2,1,2)
stairs(rx_bits(1:40),'LineWidth',2)
grid on
title('Received Bits (first 40)')
ylim([-0.5 1.5])


%% I and Q Symbols

figure;

subplot(2,1,1)
stem(I(1:40),'filled')
grid on
title('Baseband In-Phase (I) Symbols')


subplot(2,1,2)
stem(Q(1:40),'filled')
grid on
title('Baseband Quadrature (Q) Symbols')

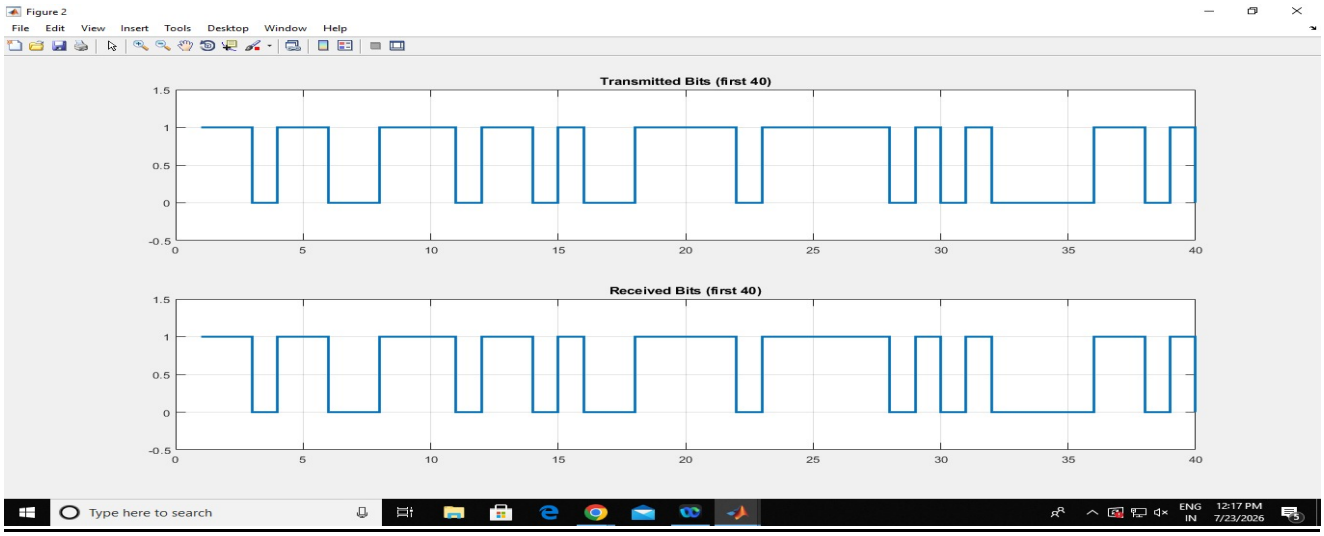

%% Waveforms

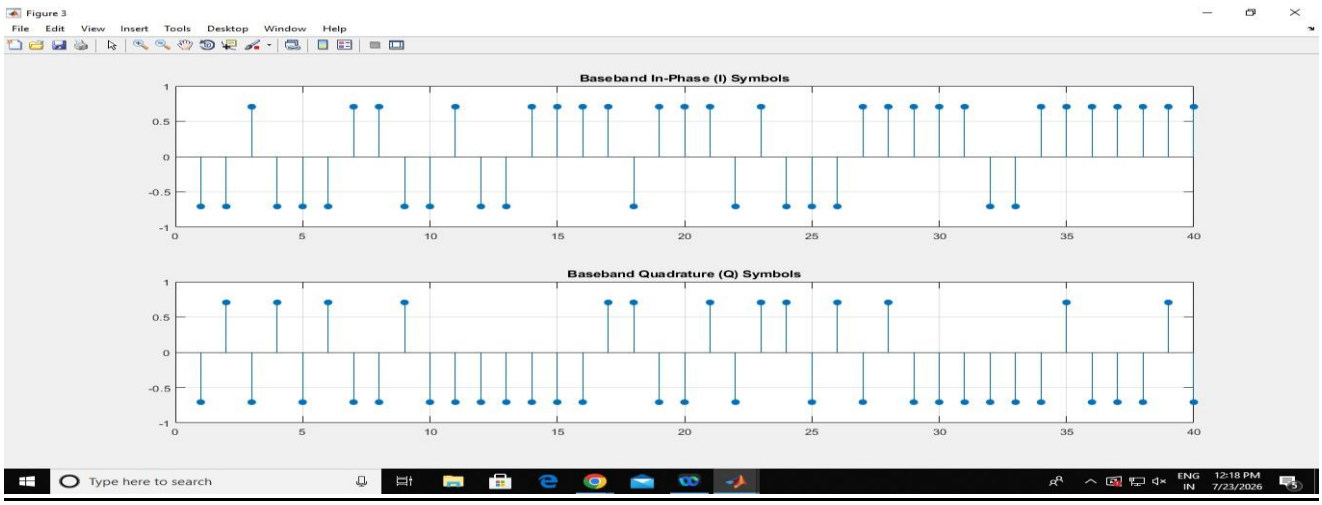
figure;
```

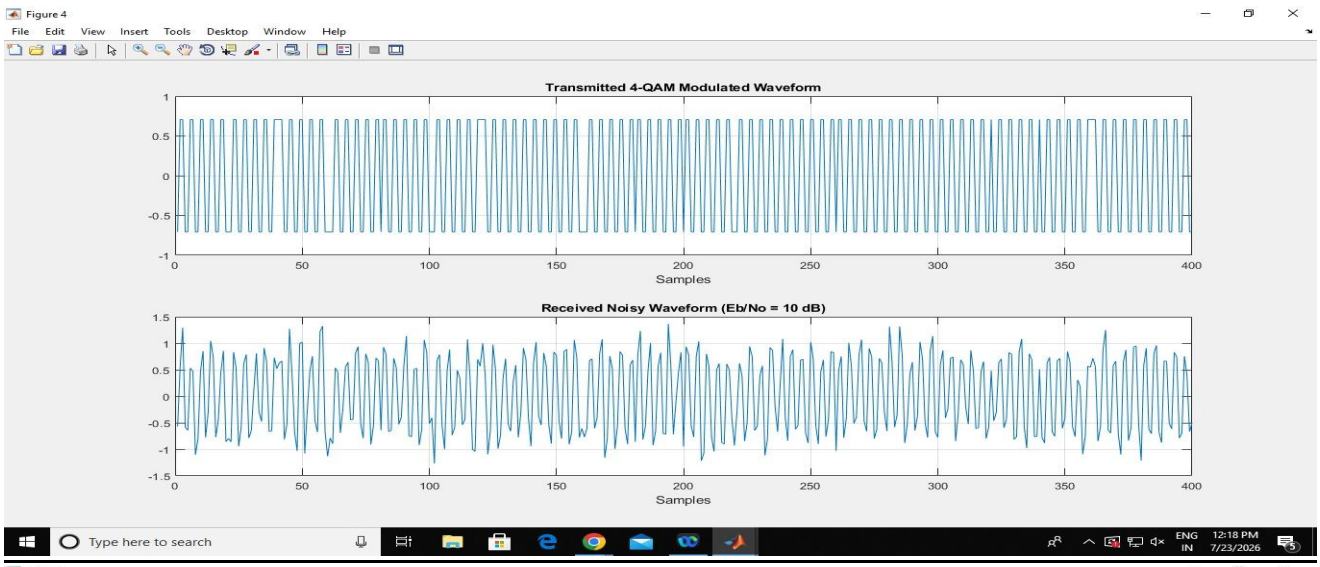
```
subplot(2,1,1)
plot(tx_signal(1:400))
grid on
title('Transmitted 4-QAM Modulated Waveform')
xlabel('Samples')

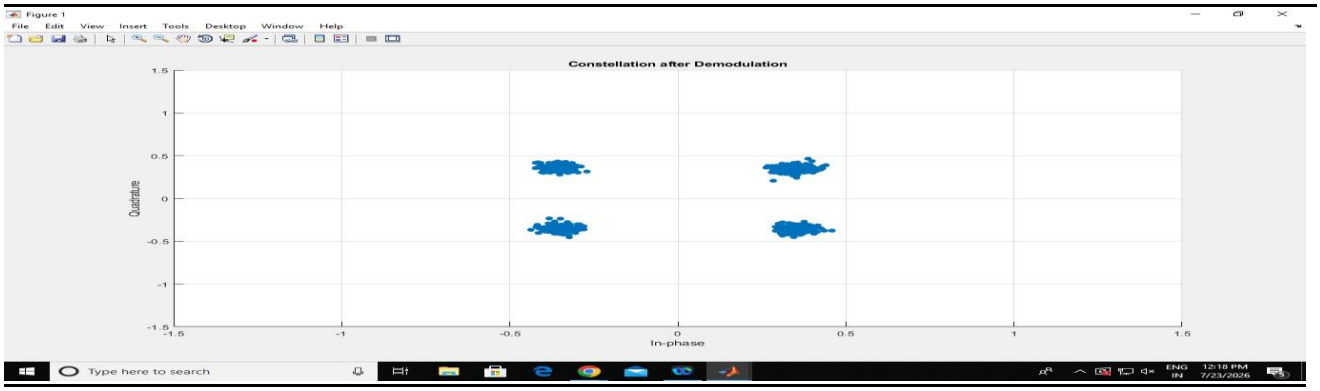

subplot(2,1,2)
plot(rx_signal(1:400))
grid on
title('Received Noisy Waveform (Eb/No = 10 dB)')
xlabel('Samples')
```

OUTPUT:









APPLICATIONS:

- 4G LTE and 5G mobile communication
- Wi-Fi (IEEE 802.11)
- Digital Cable Television
- Broadband Internet (DSL)
- Satellite Communication

ADVANTAGES:

- High data transmission rate.
- Better bandwidth efficiency.
- Suitable for high-speed communication systems.

- Widely used in modern wireless communication.

DISADVANTAGES:

- More sensitive to noise than PSK.
- Requires a higher Signal-to-Noise Ratio (SNR).
- Receiver design is more complex.
- Performance degrades in fading channels.

RESULT:

The 16-QAM signal was successfully generated and detected using MATLAB. The transmitted symbols were correctly recovered after demodulation, and the constellation diagram verified the proper implementation of 16-QAM modulation and demodulation.

EXPT.NO: 10	SIMULATION OF LINEAR BLOCK AND CYCLIC ERROR CONTROL CODING SCHEMES
DATE:	

A)Linear Block Error Control Coding Using MATLAB

AIM:

To implement a Linear Block Coding scheme for error detection and correction using MATLAB.

HARDWARE/SOFTWARE REQUIREMENTS:

- Personal Computer (PC) or Laptop
- MATLAB (R2018 or later)
- Windows/ Linux Operating system

THEORY:

Linear Block Codes are error-control codes in which k information bits are encoded into n-bit codewords using a generator matrix (G). The encoded codewords are transmitted through the communication channel. At the receiver, a parity-check matrix (H) is used to detect and correct errors.

Example: (7,4) Linear Block Code

Number of message bits (k) = 4
Number of code bits (n) = 7

PROCEDURE:

1. Switch on the computer and launch the MATLAB software.

2. Create a new script file by selecting Home → New Script.

3. Enter the MATLAB program for Linear Block Coding in the editor window.

4. Define the input message bits to be encoded.

5. Define the Generator Matrix (G) corresponding to the chosen linear block code, such as the (7,4) code.

6. Encode the message by multiplying the message vector with the generator matrix and performing modulo-2 arithmetic using the mod() function.

7. Display the encoded codeword in the MATLAB Command Window.

8. (Optional) Introduce an error in the codeword to study the error detection capability.

9. Save the script with a suitable name, such as linear_block.m.

10. Execute the program by clicking the Run button or pressing F5.

11. Observe the encoded codeword displayed in the Command Window.

12. Verify that the codeword has the correct number of parity bits and record the output.

13. Note the observations and conclude the experiment.

MATLAB CODE:

```
clc;
clear;
close all;

%% Input Data
data = [1 0 1 1 0 1 0 0];

disp('Original Data:')
disp(data)


%% Generator Matrix (7,4)

G = [1 0 0 0 1 1 0;
    0 1 0 0 1 0 1;
    0 0 1 0 1 1 1;
    0 0 0 1 0 1 1];


%% Encoding
```

```
m = reshape(data,4,[])';

code = mod(m*G,2);

tx = reshape(code',1,[]);

disp('Encoded Data:')
disp(tx)


%% Add Single Bit Error

rx = tx;

error_position = 3;      % Change any position from 1 to 14

rx(error_position)=xor(rx(error_position),1);


disp('Received Data:')
disp(rx)



%% Parity Check Matrix

H = [1 0 1 0 1 0 1;
    0 1 1 0 0 1 1;
    0 0 0 1 1 1 1];


%% Syndrome Calculation

rx_matrix = reshape(rx,7,[])';

decoded=[];


for i=1:size(rx_matrix,1)

   r = rx_matrix(i,:);
```

```
    syndrome = mod(H*r',2);

    disp('Syndrome:')
    disp(syndrome')


    % Error location

    error_pos=0;

    for j=1:7
        if isequal(H(:,j),syndrome)
            error_pos=j;
        end
    end


    % Correct error

    if error_pos~=0
        r(error_pos)=xor(r(error_pos),1);
        disp(['Error corrected at position ',num2str(error_pos)])
    else
        disp('No error detected')
    end


    decoded=[decoded r(1:4)];

end


disp('Decoded Data:')
disp(decoded)
```

OUTPUT:

Original Data:

1 0 1 1 0 1 0 0

Encoded Data:

1 0 1 1 0 1 0 0 1 0 0 1 0 1

Received Data:

1 0 0 1 0 1 0 0 1 0 0 1 0 1

Syndrome:

1 1 0

Error corrected at position 3

Syndrome:

0 0 0

No error detected

Decoded Data:

1 0 1 1 0 1 0 0

APPLICATIONS:

- Satellite communication
- Data storage devices
- Digital communication
- Computer networks

ADVANTAGES:

- Simple encoding and decoding.
- Detects and corrects errors.
- Improves communication reliability.

DISADVANTAGES:

- Lower coding efficiency than convolutional codes.
- Limited error correction capability.

RESULT:

The Linear Block Coding scheme was successfully implemented using MATLAB, and the encoded codeword was generated correctly.

B)CYCLIC ERROR CONTROL CODING USING MATLAB

AIM:

To implement a Cyclic Error Control Coding scheme using MATLAB.

HARDWARE/SOFTWARE REQUIREMENTS:

- Personal Computer (PC) or Laptop
- MATLAB (R2018 or later)
- Windows/ Linux Operating system

THEORY:

A Cyclic Code is a special type of linear block code in which every cyclic shift of a valid codeword is also a valid codeword. Cyclic codes are generated using a generator polynomial and are widely used for error detection in communication systems.

Example:

Message = 1011

Generator Polynomial = 1011

PROCEDURE:

1. Switch on the computer and open MATLAB.

2. Create a new script file by selecting Home → New Script.

3. Enter the MATLAB program for Cyclic Error Control Coding.

4. Define the input message bits.

5. Define the generator polynomial for cyclic encoding.

6. Append zeros to the message equal to the degree of the generator polynomial.

7. Perform modulo-2 division between the extended message and the generator polynomial using XOR operations.

8. Obtain the remainder generated after the division process.

9. Append the remainder to the original message to form the cyclic codeword.

10. Save the script with a suitable name, such as cyclic_code.m.

11. Execute the program by clicking the Run button or pressing F5.

12. Observe the message bits, remainder, and generated cyclic codeword displayed in the Command Window.

13. Verify that the generated codeword satisfies the cyclic coding rule and record the output.

14. Note the observations and conclude the experiment.

MATLAB CODE:

```
clc;
clear;
close all;

%% Input Data
data = [1 0 1 1];

disp('Original Data:')
disp(data)


%% Cyclic Encoder (7,4)

% Generator polynomial g(x)=x^3+x+1
g = [1 0 1 1];

% Append zeros (degree of generator = 3)
msg = [data 0 0 0];

% Modulo-2 division
temp = msg;

for i = 1:length(data)
```

```
    if temp(i)==1
        temp(i:i+3)=xor(temp(i:i+3),g);
    end

end

% Remainder
remainder = temp(end-2:end);

% Codeword
codeword = [data remainder];


disp('Generated Cyclic Codeword:')
disp(codeword)



%% Introduce Error

received = codeword;

error_position = 5;      % Change error position

received(error_position)=xor(received(error_position),1);


disp('Received Codeword with Error:')
disp(received)



%% Syndrome Calculation

temp = received;

for i = 1:length(data)

    if temp(i)==1
        temp(i:i+3)=xor(temp(i:i+3),g);
    end
```

```
end

syndrome = temp(end-2:end);


disp('Syndrome:')
disp(syndrome)



%% Error Detection

if any(syndrome)

   disp('Error Detected in Received Codeword')

else

   disp('No Error Detected')

end



%% Error Correction (Single Bit)

corrected = received;

if any(syndrome)

   corrected(error_position)=xor(corrected(error_position),1);

end


disp('Corrected Codeword:')
disp(corrected)



%% Decoded Data

decoded = corrected(1:4);
```

```
disp('Decoded Data:')
disp(decoded)



%% Verification

if isequal(data,decoded)

   disp('Transmission Successful - Error Corrected')

else

   disp('Error Remains')

end
```

OUTPUT:

```
Original Data:
   1    0    1    1

Generated Cyclic Codeword:
   1    0    1    1    0    0    0

Received Codeword with Error:
   1    0    1    1    1    0    0

Syndrome:
   1    0    0

Error Detected in Received Codeword
Corrected Codeword:
   1    0    1    1    0    0    0

Decoded Data:
   1    0    1    1

Transmission Successful - Error Corrected
```

APPLICATIONS:

- CRC (Cyclic Redundancy Check)
- Ethernet
- USB communication
- Storage devices
- Digital communication systems

ADVANTAGES:

- Excellent error detection capability.
- Easy hardware implementation.
- Efficient for burst-error detection.

DISADVANTAGES:

- Limited error correction capability.
- Requires additional redundancy bits.
- Cannot correct all multiple-bit errors.

RESULT:

The Cyclic Error Control Coding scheme was successfully implemented using MATLAB. The generated cyclic codeword verified the correct implementation of cyclic encoding using the specified generator polynomial.

EXPT.NO: 11	SIMULATION OF CONVOLUTION CODING SCHEME
DATE:	

AIM:

To implement a Convolutional Coding scheme using MATLAB and generate the encoded output for a given binary input sequence.

HARDWARE/SOFTWARE REQUIREMENTS:

- Personal Computer (PC) or Laptop
- MATLAB (R2018 or later)
- Windows/ Linux Operating system

THEORY:

Convolutional coding is a Forward Error Correction (FEC) technique used in digital communication systems to improve the reliability of data transmission over noisy channels. Unlike linear block codes, convolutional codes process the input data as a continuous stream rather than fixed-size blocks. Each output bit depends not only on the current input bit but also on a number of previous input bits stored in memory elements (shift registers).
A convolutional encoder is characterized by:

Code Rate (R): Ratio of input bits to output bits (e.g., 1/2).

Constraint Length (K): Number of stages in the shift register.

Generator Polynomials: Determine how the output bits are generated.

For a rate 1/2 convolutional encoder with constraint length K = 3 and generator polynomials (7,5) in octal:

Each input bit produces two output bits. The encoder uses two memory elements (shift registers).The encoded sequence contains redundant information that enables error correction at the receiver using decoding algorithms such as the Viterbi algorithm.

Convolutional coding is widely used in satellite communication, mobile communication, Wi-Fi, deep-space communication, and digital broadcasting due to its excellent error correction capability.

PROCEDURE:

1. Switch on the computer and open MATLAB.

2. Create a new script by selecting Home → New Script.

3. Enter the MATLAB program for convolutional encoding.

4. Define the binary input sequence to be encoded.

5. Create the trellis structure using the poly2trellis() function with the required constraint length and generator polynomials.

6. Encode the input sequence using the convenc() function.

7. Display the input sequence and encoded output in the MATLAB Command Window.

8. Save the program with a suitable name (e.g., convolutional_code.m).

9. Execute the program by clicking Run or pressing F5.

10. Observe the encoded output generated by MATLAB.

11. Verify that the encoded sequence contains twice the number of bits as the input sequence (for a rate 1/2 encoder).

12. Record the output and conclude the experiment.

MATLAB CODE:

```
clc;
clear;
close all;

%% Input Binary Message

msg = [1 0 1 1 0 1 0 0];
```

```
disp('Input Binary Message:')
disp(msg)


%% Convolutional Encoder Parameters

K = 3;              % Constraint length

G1 = [1 1 1];       % Generator polynomial
G2 = [1 0 1];

disp('Code Rate:')
disp('1/2')


%% Initial Shift Register

shift_register = [0 0 0];

disp('Initial Shift Register:')
disp(shift_register)


%% Encoding

encoded = [];

disp(' ')
disp('Shift Register Status and Output')
disp('--------------------------------')


for i = 1:length(msg)

   % Shift operation
   shift_register = [msg(i) shift_register(1:K-1)];


   % Modulo-2 addition
   out1 = mod(sum(shift_register.*G1),2);
   out2 = mod(sum(shift_register.*G2),2);
```

```
    % Store encoded bits
    encoded = [encoded out1 out2];


    % Display

    fprintf('Input Bit = %d\n',msg(i));

    fprintf('Shift Register = ');
    disp(shift_register);

    fprintf('Encoded Output = %d %d\n\n',out1,out2);

end


%% Final Results

disp('Encoded Output Sequence:')
disp(encoded)

disp('Code Rate:')
disp('1/2')
```

OUTPUT:

Input Binary Message:

1 0 1 1

Code Rate:

1/2

Constraint Length:

K = 3

Generator Polynomials:

G1 = 111
G2 = 101

Initial Shift Register:

0 0 0

Shift Register Status and Output

Input Bit = 1

Shift Register = 1 0 0

Encoded Output = 1 1

Input Bit = 0

Shift Register = 0 1 0

Encoded Output = 1 0

Input Bit = 1

Shift Register = 1 0 1

Encoded Output = 0 0

Input Bit = 1

Shift Register = 1 1 0

Encoded Output = 0 1

Encoded Output Sequence:

1 1 1 0 0 0 0 1

Code Rate:

1/2

APPLICATIONS:

- Mobile communication (3G, 4G, 5G)
- Satellite communication
- Wi-Fi (IEEE 802.11)
- Deep-space communication
- Digital television (DVB)
- Wireless communication systems

ADVANTAGES:

- Provides excellent error correction capability.
- Improves communication reliability in noisy channels.
- Suitable for continuous data transmission.
- Widely used in modern digital communication systems.

DISADVANTAGES:

- Requires more bandwidth due to redundancy.
- Decoding is computationally complex (e.g., Viterbi decoder).
- Memory and processing requirements increase with constraint length.

RESULT:

The convolutional coding scheme was successfully implemented using MATLAB. The input binary sequence was encoded using a rate 1/2 convolutional encoder with constraint length 3, and the encoded output was obtained successfully, demonstrating the effectiveness of convolutional coding for error control in digital communication systems.